

Universidad Mariano Gálvez de Guatemala
Campus Huehuetenango
Facultad de Ingeniería en Sistema
Catedrático: Lcda. Mildred Veronica Palacios Martinez
Curso: Automatas y Lenguajes Formales Año: 2026
Plan: Diario

SISTEMA DE GENERACIÓN Y VALIDADOR LÉXICO DE CREDENCIALES SEGURAS

Cristian Antonio Mendoza Mendoza 4490-24-10243

Freidh Yannel Morales 4490-24

Walter Emanuel Figueroa Vásquez 4490-24

Huehuetenango, Guatemala 2026

2. Investigación del estándar de contraseñas seguras (NIST SP 800-63B + CIS)

Qué son estos dos estándares y por qué se usan juntos

NIST SP 800-63B, titulado *Digital Identity Guidelines: Authentication and Lifecycle Management*, es la publicación especial del National Institute of Standards and Technology (Estados Unidos) que define los requisitos técnicos que debe cumplir un sistema de autenticación: cómo deben construirse las contraseñas, cómo debe verificarlas el servidor, cómo deben almacenarse y qué está prohibido exigirle al usuario. Es el documento que la industria toma hoy como referencia normativa, y su edición vigente es la Revisión 4, publicada en 2025.

El CIS Password Policy Guide, del Center for Internet Security, no compite con NIST sino que lo traduce a una política operativa concreta. Mientras NIST enuncia requisitos con verbos normativos (SHALL, SHOULD, MAY), CIS entrega una tabla de valores listos para configurar en un directorio activo o en una aplicación: longitud mínima, historial de contraseñas, umbral de bloqueo de cuenta, ventana de reintentos. Por eso la tarea pide ambos: NIST aporta el fundamento y la justificación, CIS aporta los números que se pueden implementar y defender.

Para un validador léxico esta distinción importa mucho: NIST prohíbe explícitamente las reglas de composición (obligar a mayúscula, número y símbolo), mientras que CIS sí las admite en cuentas sin segundo factor. Un proyecto académico de autómatas necesita reglas verificables por patrón, así que lo razonable es construir la política del sistema sobre la base de NIST y tomar de CIS las reglas de composición que sí pueden expresarse con expresiones regulares.

Terminología indispensable

Término	Significado en el estándar
Password / contraseña	Valor secreto elegido por el suscriptor o generado aleatoriamente por el proveedor. En la Rev. 4 NIST abandona el término *memorized secret* y usa directamente *password*. Es el factor «algo que sabes».
Passphrase / frase de paso	Contraseña larga formada por varias palabras, preferentemente no relacionadas entre sí. Puede contener espacios. Es la forma recomendada de alcanzar longitudes altas sin sacrificar memorabilidad.
Verifier / verificador	El componente del sistema que recibe la contraseña y decide si es válida. Casi todos los requisitos del estándar recaen sobre él, no sobre el usuario.
CSP	Credential Service Provider: la entidad que emite y administra las credenciales del suscriptor.
Blocklist	Lista de contraseñas prohibidas: filtraciones conocidas, palabras de diccionario y términos del contexto (nombre del servicio, nombre de usuario).
Entropía	Medida en bits de la incertidumbre de una contraseña frente a un atacante. Se calcula como log2 del espacio de búsqueda; crece linealmente con la longitud y solo logarítmicamente con el tamaño del alfabeto.
AAL	Authentication Assurance Level. Niveles 1, 2 y 3 que gradúan la fortaleza del proceso de autenticación completo, no solo de la contraseña.
SHALL / SHOULD / MAY	Verbos normativos: obligatorio, recomendado y permitido respectivamente. SHALL NOT es prohibición estricta.

2.1 Requisitos de longitud según NIST SP 800-63B Rev. 4

La longitud es el único parámetro que NIST considera determinante de la fortaleza. La Revisión 4 endureció el mínimo respecto de la versión de 2017, que pedía 8 caracteres en todos los casos:

- Mínimo obligatorio de 15 caracteres cuando la contraseña es el único factor de autenticación. Es un requisito SHALL, no una recomendación.
- Mínimo obligatorio de 8 caracteres cuando la contraseña se usa solo como parte de un proceso multifactor. El verificador puede permitir esa longitud menor, pero nunca menos de 8.
- Máximo: el verificador debería permitir al menos 64 caracteres. Este límite existe para habilitar frases de paso y gestores de contraseñas.
- La contraseña debe verificarse completa: está prohibido truncarla o pedir solo un subconjunto de sus caracteres.
- Cada punto de código Unicode cuenta como un solo carácter al medir la longitud.

Por qué la longitud pesa más que la complejidad. El espacio de búsqueda de una contraseña es A^L, donde A es el tamaño del alfabeto y L la longitud. Pasar de un alfabeto de 26 letras a uno de 94 caracteres visibles multiplica la entropía por un factor constante ($\log_2 94 / \log_2 26 \approx 1.39$), pero agregar caracteres la multiplica por L. Una frase de 20 letras minúsculas ofrece unos 94 bits de espacio bruto; una contraseña de 8 caracteres con los 94 visibles ofrece unos 52 bits. La longitud gana por amplio margen, y además no obliga al usuario a inventar sustituciones memorísticas.

Conjunto de caracteres aceptado

El estándar obliga al sistema a ser permisivo con el alfabeto, no restrictivo:

- Deberían aceptarse todos los caracteres ASCII imprimibles (RFC 20), es decir el rango de 0x20 a 0x7E, incluido el espacio.
- Deberían aceptarse también caracteres Unicode (ISO/IEC 10646). Si se aceptan, el verificador debería aplicar normalización NFC antes de calcular el hash, para que el mismo texto escrito en distintos teclados produzca el mismo resultado.
- El verificador puede tolerar errores de tecleo menores, como eliminar espacios al inicio y al final, siempre que la contraseña siga cumpliendo la longitud mínima y no se reduzca significativamente su complejidad.

Este punto tiene consecuencia directa en el diseño del analizador léxico: el alfabeto Σ del autómata que valide contraseñas debe cubrir como mínimo los 94 caracteres ASCII visibles (0x21 a 0x7E), no solo letras y dígitos. El espacio (0x20) se descarta antes de leer, porque el enunciado del proyecto indica que los espacios no se toman en cuenta.

Categoría	Rango ASCII	Caracteres
Mayúsculas	0x41–0x5A	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Minúsculas	0x61–0x7A	a b c d e f g h i j k l m n o p q r s t u v w x y z
Dígitos	0x30–0x39	0 1 2 3 4 5 6 7 8 9
Especiales	varios	ver lista abajo
Espacio	0x20	se descarta al leer: el enunciado indica que los espacios no se toman en cuenta

Especiales:

```
! " # $ % & ' ( ) * + , - . / : ; < = > ? @ [ \ ] ^ _ ` { | } ~
```

2.2 Reglas de composición: lo que NIST prohíbe exigir

Este es el cambio conceptual más importante y el que más suele sorprender. NIST establece que los verificadores NO deben imponer reglas de composición, es decir, no deben exigir mezclas de tipos de caracteres. La única razón admitida para rechazar una contraseña elegida por el usuario es que aparezca en la blocklist.

La justificación aparece en el apéndice del propio estándar: el análisis de bases de datos de contraseñas filtradas demostró que el beneficio de esas reglas es mucho menor de lo que se suponía, mientras que el impacto en usabilidad y memorabilidad es severo. En la práctica, obligar a una mayúscula y un número produce patrones predecibles del tipo `Primera1!` que los atacantes ya incorporaron a sus diccionarios.

En la misma línea, el estándar prohíbe otras dos prácticas heredadas: almacenar pistas de contraseña accesibles sin autenticación, y usar preguntas de seguridad o autenticación basada en conocimiento (el nombre de tu primera mascota) como parte de la elección de contraseñas.

Matiz para el proyecto: CIS sí conserva una regla mínima de composición para cuentas sin MFA (al menos un carácter no alfabético), precisamente porque en muchos entornos reales no se puede garantizar el cribado contra listas de filtraciones. Como un validador léxico académico no tiene acceso a una base de contraseñas comprometidas, adoptar la regla de composición de CIS es defendible y además produce reglas expresables como expresiones regulares, que es justo lo que el curso exige.

Cribado contra listas de contraseñas comprometidas (blocklist)

Al crear o cambiar una contraseña, el verificador debe compararla contra una lista de valores conocidos como comunes, esperables o comprometidos. La comparación se hace sobre la contraseña completa, no sobre subcadenas o palabras contenidas en ella. La lista debe incluir, entre otros:

- Contraseñas obtenidas de filtraciones previas (*breach corpuses*).
- Palabras de diccionario.
- Palabras específicas del contexto: el nombre del servicio, el nombre de usuario y sus derivados.

Si la contraseña propuesta aparece en la lista, el sistema debe exigir otra y explicar el motivo del rechazo, además de ofrecer orientación para elegir una fuerte; esto último evita que el usuario responda con una modificación trivial de la contraseña rechazada. El estándar advierte que listas excesivamente grandes aportan poco beneficio adicional, porque la defensa contra ataques en línea ya está cubierta por la limitación de intentos.

Rotación periódica: prohibida salvo evidencia de compromiso

El verificador no debe obligar a cambiar la contraseña de forma periódica. Solo debe forzar el cambio cuando exista evidencia de que el autenticador fue comprometido. El fundamento es empírico: cuando el usuario sabe que tendrá que cambiar la contraseña pronto, elige una más débil, y al llegar el momento del cambio aplica una transformación predecible (incrementar un número, cambiar la estación del año). Esto genera una falsa sensación de seguridad, porque un atacante que obtuvo una contraseña anterior puede aplicar esas mismas transformaciones.

Otro marco se alinea solo en parte con esta postura: el requisito 8.3.9 de PCI DSS v4.0.1 exige, cuando la contraseña es el único factor de autenticación, cambiarla al menos cada 90 días, o bien analizar de forma dinámica la seguridad de la cuenta y decidir el acceso en tiempo real; el requisito no aplica donde se usa

autenticación multifactor. El requisito 8.3.6 pide al menos 12 caracteres (8 si el sistema no admite 12) con letras y números.

Almacenamiento seguro: salt y hashing

El estándar dedica requisitos específicos a cómo se guarda la contraseña, porque de nada sirve una política de creación fuerte si la base de datos filtrada permite recuperar los secretos:

- Las contraseñas deben almacenarse en una forma resistente a ataques offline: con salt y hasheadas mediante un esquema de hashing de contraseñas adecuado.
- El salt debe tener al menos 32 bits y elegirse de modo que se minimicen las colisiones entre hashes almacenados; se guardan tanto el salt como el hash resultante.
- Debe registrarse también la referencia al esquema y al factor de costo usados, para permitir migraciones futuras de algoritmo.
- El factor de costo debe ser tan alto como sea práctico sin degradar el rendimiento, y debe incrementarse con el tiempo conforme aumenta la capacidad de cómputo.
- Se recomienda una iteración adicional de cifrado o hashing con clave (*pepper*) usando una clave secreta conocida solo por el verificador y almacenada por separado, idealmente en un HSM o entorno protegido por hardware.
- La transmisión de la contraseña debe realizarse siempre por un canal protegido y autenticado, con criptografía aprobada.

En términos prácticos esto descarta MD5, SHA-1 y SHA-256 a secas para almacenar contraseñas, y apunta a esquemas con factor de trabajo ajustable como PBKDF2, bcrypt, scrypt o Argon2id.

Limitación de intentos (rate limiting) y controles de cuenta

La política de creación se complementa con controles de tiempo de ejecución. El verificador debe limitar los intentos fallidos consecutivos sobre una misma cuenta a no más de 100, deshabilitando el autenticador al alcanzar ese tope; ese valor es una cota superior y cada organización puede fijar límites menores. El estándar sugiere además técnicas para evitar que un atacante bloquee al usuario legítimo: retardos crecientes tras cada fallo (desde 30 segundos hasta una hora), desafíos de detección de bots y autenticación adaptativa basada en riesgo, considerando dirección IP, geolocalización y patrones de uso.

CIS es más concreto y propone bloqueos por umbral de intentos con una ventana de reinicio definida y alerta a personal clave cuando se supera el límite, precisamente porque un directorio corporativo necesita valores configurables.

Requisitos de usabilidad

La Revisión 4 convierte en normativos varios puntos que antes eran buenas prácticas, todos orientados a que el usuario pueda usar contraseñas largas sin fricción:

- El verificador debe permitir el uso de gestores de contraseñas y de la funcionalidad de autocompletado.
- Debería permitirse la función de pegar en el campo de contraseña; bloquearla rompe los gestores y empuja al usuario a elegir contraseñas más cortas.

- Debería ofrecerse la opción de mostrar la contraseña en claro mientras se escribe, para que el usuario confirme lo que tecleó; opcionalmente puede mostrarse cada carácter durante un instante tras pulsarlo.
- Debe entregarse orientación al usuario para elegir una contraseña fuerte, especialmente después de un rechazo por blocklist.

El CIS Password Policy Guide

La guía de CIS parte de una jerarquía explícita de métodos de autenticación, de mejor a peor: primero MFA, que debería ser la meta para toda autenticación de usuarios; segundo gestores de contraseñas, que generan y almacenan contraseñas únicas y largas por cuenta; y tercero las contraseñas generadas y recordadas por humanos, caso al que está dirigida la mayor parte de la guía porque no va a desaparecer pronto.

Su recomendación central sobre longitud distingue dos escenarios y es la que suele citarse en las políticas corporativas:

Control	Cuenta solo con contraseña	Cuenta con MFA (contraseña como factor)
Longitud mínima	14 caracteres; enseñar y fomentar el uso de frases de paso	8 caracteres
Longitud máxima	Tan larga como el sistema permita; no fijarla por política	Igual
Composición	Al menos 1 carácter no alfabético	Sin requisito de composición
Tipos de caracteres	Permitir todos los tipos, sin restricciones	Igual
Historial de contraseñas	Las últimas 5 o más	Igual
Retardo entre cambios	1 día o más	Igual
Expiración periódica	Cambio inmediato ante un evento (rotación de personal, brecha); respaldo anual como máximo	Igual
Lista de bloqueo	Verificar contra listas de contraseñas filtradas o prohibidas	Igual

La guía también insiste en prohibir información fácilmente adivinable, como el nombre de usuario o el nombre de la organización, y en un punto didáctico que conviene rescatar: una frase de paso larga sigue siendo razonable aunque contenga una palabra común, porque esa palabra representa una fracción pequeña del total de caracteres desconocidos. Esto explica por qué CIS prefiere enseñar frases de paso antes que imponer complejidad.

Comparación directa entre ambos estándares

Aspecto	NIST SP 800-63B Rev. 4	CIS Password Policy Guide
Longitud mínima sin MFA	15 caracteres (obligatorio)	14 caracteres
Longitud mínima con MFA	8 caracteres	8 caracteres
Longitud máxima	Permitir al menos 64	La máxima que soporte el sistema
Reglas de composición	Prohibidas	1 carácter no alfabético si no hay MFA
Expiración periódica	Prohibida salvo compromiso	No forzada por calendario, salvo respaldo anual; cambio inmediato ante compromiso
Blocklist	Obligatoria, sobre la contraseña completa	Recomendada y continua
Historial	No lo especifica	Últimas 5 o más
Bloqueo de cuenta	Máximo 100 intentos fallidos consecutivos	Umbral configurable con ventana de reinicio y alerta
Enfoque	Normativo: qué debe cumplir el verificador	Operativo: qué valores configurar

2.3 Política consolidada propuesta para el sistema

De la lectura combinada de ambos estándares se deriva la siguiente política, que es la que el generador y el validador léxico del proyecto deberían implementar. Se separa deliberadamente lo que es verificable por patrón (y por tanto expresable con expresiones regulares y autómatas finitos) de lo que requiere consultar datos externos.

Reglas verificables léxicamente

N.º	Regla	Fundamento
R1	Longitud mínima de 15 caracteres	NIST: mínimo obligatorio para contraseña como único factor
R2	Longitud máxima de 64 caracteres	NIST: el verificador debe permitir al menos 64
R3	Alfabeto: los 94 caracteres ASCII visibles (0x21 a 0x7E); el espacio no se toma en cuenta	NIST: aceptar ASCII imprimible; el enunciado del proyecto indica que los espacios no se toman en cuenta
R4	Al menos una letra minúscula	Complemento CIS para cuentas sin MFA
R5	Al menos una letra mayúscula	Complemento CIS para cuentas sin MFA
R6	Al menos un dígito	CIS: al menos un carácter no alfabético
R7	Al menos un carácter especial	Refuerzo de la regla anterior de CIS
R8	Los espacios se descartan antes de validar (no forman parte del alfabeto)	Enunciado del proyecto; NIST además permite recortar espacios en los extremos
R9	Sin tres o más caracteres idénticos consecutivos	Evita patrones triviales del tipo aaa o 111. CIS §5.1.4: caracteres repetitivos o secuenciales
R10	Sin secuencias ascendentes o descendentes de 4 símbolos consecutivos (abcd, dcba, 1234, 4321), sin distinguir mayúsculas	CIS §5.1.4: caracteres secuenciales (1234abcd). Palabras como qwerty se tratan como palabras prohibidas (P8, paso 4.b)

Reglas no verificables léxicamente (requieren datos o estado)

- Comparación contra la blocklist completa de contraseñas filtradas y palabras de diccionario (millones de entradas que cambian con el tiempo). Una lista finita y fija de palabras prohibidas

(por ejemplo password, admin o qwerty) sí es un lenguaje regular y se implementa como regla P8 en el paso 4.

- Rechazo de términos que dependen del contexto en tiempo de ejecución: nombre de usuario y derivados (varían con cada cuenta).
- Historial de las últimas 5 contraseñas del usuario.
- Almacenamiento con salt y hashing de costo ajustable.
- Limitación de intentos fallidos y bloqueo de cuenta.

Esta separación es exactamente el límite de expresividad de los lenguajes regulares: un autómata finito no tiene memoria para consultar una lista externa ni para comparar con contraseñas anteriores, pero sí puede decidir pertenencia a un lenguaje definido por longitud, alfabeto y presencia de clases de caracteres. Por la misma razón, una lista finita y fija de palabras sí puede reconocerse con un autómata; lo que no puede es consultar una base externa y cambiante.

Formalización: expresión regular y gramática

La política de las reglas R1 a R7 se expresa habitualmente con una expresión regular con anticipaciones (*lookaheads*), disponible en motores como PCRE, Java, .NET, Python y JavaScript:

```
^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!"#$%&'()*+,\-./:;<=>?@\[\\\]^_`{|}~])
[\x20-\x7E]{15,64}$
```

Nota: esta expresión con anticipaciones se incluye solo como referencia comparativa con los motores de expresiones regulares (PCRE, Python, JavaScript). Las anticipaciones no pertenecen al formalismo de ER que exige el punto 4 (unión, concatenación y cierre de Kleene), y su rango `[\x20-\x7E]` incluye el espacio, que el proyecto descarta. La ER formal, sobre el alfabeto de 94 símbolos, se desarrolla en el paso 4 (4.d).

Conviene aclarar un punto teórico que el curso valorará: las anticipaciones no forman parte del formalismo regular clásico y por sí solas podrían sugerir que el lenguaje no es regular. No es el caso. El lenguaje descrito sigue siendo regular, porque se trata de la intersección finita de lenguajes regulares (el de las cadenas que contienen al menos una minúscula, el de las que contienen al menos una mayúscula, etc.) con un lenguaje de longitud acotada, y los lenguajes regulares son cerrados bajo intersección. Las anticipaciones son solo azúcar sintáctica que evita escribir a mano el autómata producto.

Ese autómata producto se construye por intersección de cuatro autómatas de dos estados, uno por cada clase de carácter exigida, junto con un contador de longitud acotado. El resultado tiene a lo sumo 2^4 = 16 combinaciones de banderas multiplicadas por 65 valores de longitud (de 0 a 64), es decir un número finito de estados, lo que confirma la regularidad. Puede describirse con una tupla:

```
M = (Q, Σ, δ, q0, F)
Σ = { caracteres ASCII visibles, 0x21 a 0x7E }        |Σ| = 94
Q = { (n, a, b, c, d) | 0 ≤ n ≤ 65 ; a, b, c, d ∈ {0,1} }
    n = longitud acumulada (65 = estado de desbordamiento)
    a = ya se vio una minúscula     c = ya se vio un dígito
    b = ya se vio una mayúscula     d = ya se vio un especial
q0 = (0, 0, 0, 0, 0)
F  = { (n, 1, 1, 1, 1) | 15 ≤ n ≤ 64 }
δ((n,a,b,c,d), x) = (min(n+1, 65),
                     a ∨ esMinuscula(x),
                     b ∨ esMayuscula(x),
                     c ∨ esDigito(x),
                     d ∨ esEspecial(x))
```

La gramática regular equivalente, en forma lineal por la derecha, se obtiene tomando un no terminal por cada estado del autómata. Para una versión simplificada que solo controle el alfabeto y la longitud mínima, la producción tiene la forma:

```
S   → x A1          para todo x ∈ Σ
A1  → x A2          para todo x ∈ Σ
...
A14 → x A15         para todo x ∈ Σ
A15 → x A16 | ε     para todo x ∈ Σ   (a partir de aquí ya es aceptable)
...
A63 → x A64 | ε
A64 → ε
```

Alcance: la tupla M y la gramática anteriores cubren solo el alfabeto, la longitud y las cuatro clases de caracteres (R1 a R7) y están escritas sobre los 94 símbolos ASCII visibles. Los filtros de inseguridad se agregan como autómatas de rechazo, y el validador final es el autómata producto de todas las reglas. La correspondencia con el paso 4 es: R1 y R2 → P1; R3 → P0; R4 a R7 → P2 a P5; R9 → P6; R10 → P7; lista finita de palabras → P8. El diseño completo, con una expresión regular por regla, está en 4.b y 4.d.

Casos de prueba sugeridos para el validador

Cadena de entrada	Resultado	Motivo
`Pl4t0n0#Guatemala2026`	Válida	21 caracteres, cumple las cuatro clases exigidas
`Mi perro come 7 Huesos!`	Válida	Los espacios se descartan (enunciado); queda «Miperrocome7Huesos!» de 19 caracteres con las cuatro clases
`Abc123!`	Inválida	Solo 7 caracteres; incumple la longitud mínima
`contrasenasegura`	Inválida	Sin mayúscula, sin dígito y sin carácter especial
`CONTRASENA2026!`	Inválida	Sin letra minúscula
`Password12345!!`	Inválida	Contiene la palabra prohibida «password» (P8) y la secuencia 2345 (P7); ambas son reglas léxicas, ver paso 4
`Aaaa1111!!!!bbbb`	Inválida	Tres o más caracteres idénticos consecutivos (aaa, 1111, !!!!, bbb)
`␣␣␣␣Segura2026#xyz`	Inválida	Al descartar los espacios quedan 14 caracteres; incumple el mínimo de 15

3. Investigación del estándar para correos electrónicos (RFC 5322)

Qué es RFC 5322 y cómo se relaciona con los demás RFC

RFC 5322, titulado *Internet Message Format* y publicado en octubre de 2008 por el IETF, define la sintaxis de los mensajes de correo electrónico en Internet. Sustituye a RFC 2822, que a su vez había sustituido al histórico RFC 822 de 1982. Su sección 3.4.1 es la que interesa para este proyecto: allí se define addr-spec, la gramática formal de una dirección de correo electrónico.

Es importante no confundirlo con los documentos vecinos, porque cada uno regula una cosa distinta y una implementación correcta necesita los tres:

Documento	Regula	Aporte al validador
RFC 5322	Formato del mensaje y sintaxis de la dirección	La gramática ABNF de addr-spec: qué caracteres y estructura son sintácticamente legales
RFC 5321 (SMTP)	Transporte del mensaje	Los límites de longitud en octetos: 64 para la parte local, 255 para el dominio, 256 para la ruta completa
RFC 1035 / 1123	Sistema de nombres de dominio	Reglas de las etiquetas del dominio: caracteres permitidos, longitud de etiqueta y prohibición de guion al inicio o final
RFC 6531 / 6532	SMTPUTF8, correo internacionalizado	Extiende la dirección a caracteres Unicode fuera de ASCII
RFC 3696	Informativo: técnicas para validar nombres y direcciones	Orientación sobre longitudes y casos frontera de la parte local

3.1 La gramática ABNF de una dirección

El estándar define la dirección en notación ABNF (RFC 5234). La regla raíz es:

```
addr-spec      = local-part "@" domain
local-part     = dot-atom / quoted-string / obs-local-part
domain         = dot-atom / domain-literal / obs-domain
dot-atom       = [CFWS] dot-atom-text [CFWS]
dot-atom-text  = 1*atext *("." 1*atext)
atext          = ALPHA / DIGIT /
                 "!" / "#" / "$" / "%" / "&" / "'" / "*" /
                 "+" / "-" / "/" / "=" / "?" / "^" / "_" /
                 "`" / "{" / "|" / "}" / "~"
quoted-string  = [CFWS] DQUOTE *([FWS] qcontent) [FWS] DQUOTE [CFWS]
qcontent       = qtext / quoted-pair
qtext          = %d33 / %d35-91 / %d93-126
quoted-pair    = ("\" (VCHAR / WSP))
domain-literal = [CFWS] "[" *([FWS] dtext) [FWS] "]" [CFWS]
dtext          = %d33-90 / %d94-126
```

En palabras llanas: una dirección es una parte local, seguida obligatoriamente del carácter arroba, seguida de un dominio. El propio estándar advierte que no deberían usarse comentarios ni espacios plegables alrededor del arroba, y que si la parte local puede escribirse como dot-atom entonces debe preferirse esa forma y no la de cadena entrecomillada.

La parte local (local-part)

Es la porción que antecede al arroba y es interpretada únicamente por el servidor de destino: ningún otro sistema debe hacer suposiciones sobre su significado. Admite dos formas.

Forma dot-atom (la habitual). Una o más secuencias de caracteres atext separadas por puntos. Los caracteres atext son las letras, los dígitos y este conjunto de símbolos especiales:

```
! # $ % & ' * + - / = ? ^ _ ` { | } ~
```

De esta definición se desprenden tres restricciones que todo validador debe implementar y que son la fuente más frecuente de errores:

- El punto no puede aparecer al inicio de la parte local (`.usuario@dominio.com` es inválido).
- El punto no puede aparecer al final de la parte local (`usuario.@dominio.com` es inválido).
- No pueden aparecer dos puntos consecutivos (`usuario..nombre@dominio.com` es inválido). La regla dot-atom-text exige al menos un atext entre puntos consecutivos.

Forma quoted-string (entrecomillada). La parte local puede encerrarse entre comillas dobles, y dentro de ellas se admiten prácticamente todos los caracteres ASCII imprimibles, incluidos el espacio, la arroba, la coma y los dos puntos. Los caracteres barra invertida y comilla doble deben escaparse con barra invertida (quoted-pair). Así, `"juan perez"@ejemplo.com` y `"a@b"@ejemplo.com` son sintácticamente válidas según RFC 5322, aunque en la práctica muchos proveedores las rechacen. El estándar recomienda explícitamente no usar esta forma cuando la parte local pueda expresarse como dot-atom.

Longitud. RFC 5322 no impone un límite de longitud a la parte local; ese límite lo fija RFC 5321 en 64 octetos. Una dirección puede ser sintácticamente válida según la gramática y aun así ser rechazada en el transporte por exceder ese tamaño, de modo que un validador completo debe comprobar ambas cosas.

El dominio (domain)

Es la porción posterior al arroba e identifica al servidor responsable de la entrega. También admite dos formas.

Forma dot-atom: nombre de dominio. Es la forma normal: una secuencia de etiquetas separadas por puntos, como `correo.universidad.edu.gt`. Aunque la gramática ABNF de RFC 5322 permite aquí los mismos caracteres atext, la práctica y las reglas del DNS (RFC 1035 y RFC 1123) restringen las etiquetas de manera considerable:

- Cada etiqueta contiene solo letras, dígitos y guiones.
- Una etiqueta no puede comenzar ni terminar con guion.
- Cada etiqueta mide entre 1 y 63 caracteres.
- El nombre completo del dominio no debe exceder 255 octetos, y RFC 5321 fija 255 como límite práctico para esta porción de la dirección.
- El dominio no distingue mayúsculas de minúsculas, a diferencia de la parte local, que formalmente sí podría distinguirlas.

Dominio de nivel superior (TLD). La última etiqueta es el dominio de nivel superior. Conviene documentar su tipología en el proyecto porque suele pedirse en la sustentación:

Tipo	Descripción	Ejemplos
gTLD genéricos	De propósito general, administrados por ICANN	.com .org .net .info
ccTLD de país	Dos letras según ISO 3166-1 alfa-2	.gt .mx .es .us
Patrocinados / restringidos	Requieren pertenecer a una comunidad	.edu .gov .mil .int
Nuevos gTLD	Creados desde 2013, de longitud variable	.tech .online .dev .app
De segundo nivel bajo ccTLD	Combinaciones jerárquicas frecuentes	.edu.gt .com.gt .gob.gt .co.uk

Para efectos del validador, el criterio práctico más aceptado es exigir que el TLD tenga al menos dos caracteres y esté formado solo por letras, aunque técnicamente existen TLD internacionalizados codificados en Punycode que comienzan con el prefijo `xn--`.

Forma domain-literal: dirección IP literal. El dominio puede sustituirse por una dirección de red encerrada entre corchetes, como `usuario@[192.168.1.10]` o `usuario@[IPv6:2001:db8::1]`. Es sintácticamente válido y se usa en entornos internos o de diagnóstico, pero prácticamente ningún servicio público lo acepta, por lo que la mayoría de validadores lo excluyen deliberadamente.

Límites de longitud aplicables

Elemento	Límite	Fuente
Parte local	64 octetos	RFC 5321, sección 4.5.3.1
Dominio	255 octetos	RFC 5321, sección 4.5.3.1
Dirección completa (path)	256 octetos	RFC 5321, sección 4.5.3.1
Etiqueta DNS individual	63 caracteres	RFC 1035
Línea de texto del mensaje	998 caracteres más CRLF	RFC 5322, sección 2.1.1

Se habla de octetos y no de caracteres porque en correo internacionalizado un mismo carácter puede ocupar varios bytes en UTF-8.

Ejemplos válidos e inválidos

Dirección	Resultado	Análisis
`cristian.lopez@ejemplo.com`	Válida	dot-atom con un punto interior; dominio con TLD de tres letras
`usuario+etiqueta@correo.edu.gt`	Válida	El signo más es atext; dominio con TLD de segundo nivel
`a_b-c'd!e@sub.dominio.org`	Válida	Todos los símbolos empleados pertenecen a atext
`"juan perez"@ejemplo.com`	Válida (poco usual)	Forma quoted-string; el espacio solo es legal entre comillas
`usuario@[192.168.1.10]`	Válida (poco usual)	domain-literal con dirección IPv4
`.usuario@ejemplo.com`	Inválida	Punto inicial: dot-atom-text exige atext antes del punto
`usuario.@ejemplo.com`	Inválida	Punto final en la parte local
`usuario..nombre@ejemplo.com`	Inválida	Puntos consecutivos sin atext entre ellos
`usuarioejemplo.com`	Inválida	Dos arrobas: la gramática admite exactamente una
`usuario@ejemplo`	Inválida en la práctica	Dominio sin punto ni TLD; la ABNF la admite pero no es entregable
`usuario@-ejemplo.com`	Inválida	Etiqueta de dominio que inicia con guion
`usuario@ejemplo..com`	Inválida	Etiqueta vacía entre dos puntos del dominio
`usuario ejemplo@correo.com`	Inválida (RFC 5322)	Espacio sin comillas en la parte local. El RFC lo invalida; el validador del paso 4 descarta los espacios (enunciado) y lo evalúa como `usuarioejemplo@correo.com`
`usuario@ejemplo.c`	Inválida en la práctica	TLD de un solo carácter

3.2 Expresiones regulares para el validador

La gramática completa de RFC 5322, con comentarios, espacios plegables y formas obsoletas, no se expresa cómodamente con una sola expresión regular: los intentos de hacerlo producen patrones de miles de caracteres, imposibles de mantener y de explicar. Por eso la práctica estándar es validar un subconjunto razonable. Se proponen tres niveles.

Nivel 1: subconjunto práctico recomendado

```
^[A-Za-z0-9!#$%&'*+/=?^_`{|}~-]+
(?:\.[A-Za-z0-9!#$%&'*+/=?^_`{|}~-]+)*
@
[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?
(?:\.[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?)*
\.[A-Za-z]{2,}$
```

Esta expresión cubre la forma dot-atom completa de atext, impide puntos iniciales, finales y consecutivos en ambos lados, impide guiones al inicio y al final de cada etiqueta, limita las etiquetas a 63 caracteres y exige un TLD alfabético de al menos dos letras. Es la versión que conviene implementar y sustentar.

Nivel 2: versión didáctica simplificada

```
^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
```

Es la expresión que aparece en la mayoría de tutoriales. Sirve para explicar la estructura en tres bloques, pero acepta casos inválidos como `usuario..nombre@dominio.com` o `usuario@-dominio.com`, por lo que no debería usarse como validador final sin advertirlo.

Nivel 3: especificación HTML5

```
^[a-zA-Z0-9.!#$%&'*+/=?^_`{|}~-]+
@[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?
(?:\.[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?)*$
```

Es la que usan los navegadores para `input type=email`. El propio estándar HTML la describe como una violación deliberada de RFC 5322: es más estricta en unos aspectos, porque excluye las formas entrecomilladas, y más laxa en otros, porque admite direcciones sin TLD como `usuario@localhost`.

3.3 Descomposición en tokens para el analizador léxico

Al implementar el validador conviene tratar la dirección como una secuencia de tokens, lo que facilita reportar errores precisos en lugar de un simple rechazo global. Ejemplos tomados de `cristian.lopez@correo.edu.gt`:

```
Token     Patrón                                          Ejemplo
ATEXT     [A-Za-z0-9!#$%&'*+/=?^_`{|}~-]+                 cristian, lopez
PUNTO     \.                                              el separador entre etiquetas
ARROBA    @                                               el separador de secciones
ETIQUETA  [A-Za-z0-9]([A-Za-z0-9-]*[A-Za-z0-9])?          correo, edu
TLD       [A-Za-z]{2,}                                    gt
```

3.4 Autómata finito determinista para validar la dirección

El subconjunto de nivel 1 es un lenguaje regular y admite un AFD. La siguiente formulación de siete estados es una versión simplificada: cubre la estructura (parte local, arroba y etiquetas del dominio) pero no verifica el dominio de nivel superior ni los límites de longitud.

Estado	Significado	Transiciones
q0	Inicial; aún no se lee nada	atext → q1 ; cualquier otro → error
q1	Dentro de la parte local, último carácter es atext	atext → q1 ; punto → q2 ; arroba → q3
q2	Se leyó un punto en la parte local	atext → q1 ; punto o arroba → error
q3	Se leyó el arroba; se espera el dominio	letra o dígito → q4 ; cualquier otro → error
q4	Dentro de una etiqueta del dominio	letra o dígito → q4 ; guion → q5 ; punto → q6
q5	Último carácter de la etiqueta fue guion	letra o dígito → q4 ; guion → q5 ; punto → error
q6	Se leyó un punto en el dominio; inicia nueva etiqueta	letra o dígito → q4 ; punto o guion → error

Limitación: q4 también acepta cadenas como `usuario@ejemplo`, sin punto ni TLD. El AFD completo, que exige al menos un punto después de la arroba y un TLD formado solo por letras (mínimo 2), se construye en el paso 4 (inciso 4.d.ii, Figura 6) con 12 estados por el método del árbol, 8 al minimizarlo.

4. Diseño de la expresión regular para el reconocimiento de correos y contraseñas seguras

Este paso sigue la numeración de la guía: 4.a alfabeto, 4.b reglas de contraseñas, 4.c reglas de correos, 4.d expresiones regulares, 4.d.i conversión a AFND con transiciones vacías (construcción de Thompson) y 4.d.ii conversión a AFD (algoritmo de Thompson por subconjuntos y método del árbol).

Decisiones de diseño

- **Lectura símbolo a símbolo.** La guía indica que cada símbolo se lee de uno en uno y que los espacios no se toman en cuenta. Por eso el espacio se descarta antes de alimentar el autómata (equivale a una transición ε sobre el espacio) y no pertenece a ningún alfabeto.
- **Una ER por regla.** Los lenguajes regulares son cerrados bajo intersección y complemento. En lugar de una sola expresión gigante, cada regla tiene su propia ER y su propio AFD; una credencial es válida cuando todas las reglas de aceptación la reconocen y ninguna regla de rechazo la reconoce (autómata producto). Así se sabe exactamente qué regla falla, como exige el punto 7.a.i.
- **Etiquetas.** Se trabaja con etiquetas por grupo de caracteres (L, l, N, S, A, H, E, P) para que las ER y los diagramas sean legibles. Cada etiqueta es un conjunto de símbolos del alfabeto y en el autómata equivale a una arista con todos los símbolos del grupo.

4.a Definición del alfabeto

Alfabeto de contraseñas

Etiqueta	Grupo	Rango	Cant.
L	Mayúsculas	A–Z	26
l	Minúsculas	a–z	26
N	Números	0–9	10
S	Especiales	ver lista abajo	32
Σp = L ∪ l ∪ N ∪ S	ASCII visible	0x21–0x7E	94

Especiales (S):

```
! " # $ % & ' ( ) * + , - . / : ; < = > ? @ [ \ ] ^ _ ` { | } ~
```

El espacio (0x20) se descarta antes de leer y no forma parte de Σp.

Alfabeto de correos

Etiqueta	Grupo	Cant.
A	Letras A–Z, a–z	52
N	Números 0–9	10
H	Guion -	1
E	Otros símbolos atext (ver lista abajo)	18
P	Punto .	1
@	Arroba	1
at = A ∪ N ∪ H ∪ E	Caracteres de RFC 5322 permitidos en la parte local	81
Σe = at ∪ P ∪ @	Alfabeto de correos	83

Otros símbolos atext (E):

```
! # $ % & ' * + / = ? ^ _ ` { | } ~
```

Quedan fuera de Σe los 11 símbolos siguientes (83 + 11 = 94 ASCII visibles), porque solo aparecen en las formas quoted-string, domain-literal y comentarios de RFC 5322, que el paso 3 recomienda evitar y que este validador excluye deliberadamente:

```
" ( ) , : ; < > [ \ ]
```

Un símbolo fuera del alfabeto se reporta como regla 0 (alfabeto) con su posición.

4.b Reglas según el estándar de contraseñas seguras

ID	Regla	Categoría de la guía	Tipo	Fundamento
P0	Todo símbolo pertenece a Σp	Caracteres	Aceptación	NIST: aceptar ASCII imprimible
P1	Longitud n con 15 ≤ n ≤ 64	Longitud	Aceptación	NIST Rev. 4: mínimo 15 sin MFA; permitir al menos 64
P2	Al menos una mayúscula (L)	Composición obligatoria	Aceptación	CIS: al menos un carácter no alfabético; se refuerza con las 4 clases
P3	Al menos una minúscula (l)	Composición obligatoria	Aceptación	Ídem
P4	Al menos un número (N)	Composición obligatoria	Aceptación	CIS
P5	Al menos un especial (S)	Composición obligatoria	Aceptación	CIS
P6	Tres o más símbolos idénticos consecutivos (aaa, 111, !!!)	Filtro de inseguridad	Rechazo	CIS §5.1.4: caracteres repetitivos o secuenciales
P7	Secuencia de 4 ascendente o descendente (abcd, dcba, 1234, 4321), sin distinguir mayúsculas	Filtro de inseguridad	Rechazo	CIS §5.1.4: caracteres repetitivos o secuenciales
P8	Contiene una palabra prohibida: password, contrasena, admin, qwerty, iloveyou, welcome, letmein (sin distinguir mayúsculas)	Filtro de inseguridad	Rechazo	CIS §5.1.4 (password, qwerty, iloveyou); NIST: blocklist obligatoria

Sobre la complejidad. La guía separa la composición obligatoria de la complejidad. En este diseño la complejidad léxica queda fijada por la exigencia de las cuatro clases (P2 a P5) sobre un alfabeto de 94 símbolos y por la longitud (P1). La medición en niveles baja, media o fuerte se calcula en la aplicación (punto 7.a.iv).

Sobre la composición. NIST desaconseja imponer reglas de composición. Se mantienen aquí porque un validador léxico académico no dispone de una base de contraseñas filtradas y la composición sí es expresable con expresiones regulares (ver paso 2).

Sobre los filtros de inseguridad. P6 a P8 son los casos no aceptados: su ER describe la cadena prohibida y el AFD que la reconoce marca la contraseña como insegura. La blocklist real de millones de entradas no es una regla léxica; P8 implementa solo una lista finita, que sí es un lenguaje regular. Password, qwerty e iloveyou aparecen como ejemplos en la sección 5.1.4 del CIS Password Policy Guide v21.12; contrasena es su equivalente en español; admin, welcome y letmein son ejemplos habituales de listas de contraseñas comunes y se incluyen por criterio propio, no por cita del estándar. La lista es ampliable sin cambiar el método: cada palabra nueva agrega una alternativa a la ER de P8.

Una contraseña es aceptada cuando cumple todas las reglas de aceptación y ninguna de rechazo:

```
Contraseña aceptada = P0 ∧ P1 ∧ P2 ∧ P3 ∧ P4 ∧ P5 ∧ ¬P6 ∧ ¬P7 ∧ ¬P8
```

4.c Reglas según el estándar de correos electrónicos

ID	Regla	Componente	Fundamento
C0	Todo símbolo pertenece a Σe	Alfabeto	RFC 5322: atext, punto, arroba
C1	Estructura local@dominio: exactamente una arroba y ambas partes no vacías	Estructura	RFC 5322 §3.4.1 (addr-spec)
C2	Parte local dot-atom: átomos de at separados por un solo punto; sin punto inicial, final ni doble	Parte local	RFC 5322 dot-atom-text
C3	Parte local de 1 a 64 símbolos	Parte local	RFC 5321 §4.5.3.1
C4	Dominio con al menos dos etiquetas separadas por punto; cada etiqueta de 1 a 63 símbolos, solo letras, números y guion, sin guion al inicio ni al final	Dominio	RFC 1035 / 1123
C5	TLD (última etiqueta) formado solo por letras, mínimo 2	Dominio de nivel superior	Criterio práctico del paso 3
C6	Dominio de 1 a 255 símbolos	Dominio	RFC 5321 §4.5.3.1

C1 se verifica sobre la cadena completa. Una vez confirmada la única arroba, el validador separa la cadena en parte local (a la izquierda) y dominio (a la derecha): C2 y C3 se aplican a la parte local; C4, C5 y C6, al dominio.

Exclusiones deliberadas (documentadas en el paso 3): parte local entrecomillada (quoted-string), dominio como dirección IP literal, comentarios y espacios plegables (CFWS), sintaxis obsoleta y correo internacionalizado con Unicode (RFC 6531).

```
Correo aceptado = C0 ∧ C1 ∧ C2 ∧ C3 ∧ C4 ∧ C5 ∧ C6
```

4.d Expresiones regulares obtenidas de las reglas de 4.b y 4.c

Notación: `|` es unión, la yuxtaposición es concatenación, `*` es cierre de Kleene, `x+` = `xx*`, `x?` = `(x|ε)` y `x{m,n}` = `x^m (x|ε)^(n−m)`. Las abreviaturas se expanden a las tres operaciones básicas antes de aplicar Thompson.

Contraseñas (Σ = L | l | N | S)

```
P1  Σ{15,64}
P2  Σ* L Σ*
P3  Σ* l Σ*
P4  Σ* N Σ*
P5  Σ* S Σ*
P6  Σ* ( aaa | bbb | ... | ~~~ ) Σ*        94 alternativas xxx, una por cada x ∈ Σ
P7  Σ* ( abcd | dcba | ... | 9876 ) Σ*      60 palabras, ver nota
P8  Σ* ( password | contrasena | ... ) Σ*    una alternativa por palabra
```

Nota sobre P7. Las 60 palabras son 23 ascendentes y 23 descendentes de letras (abcd … wxyz y dcba … zyxw) más 7 y 7 de dígitos (0123 … 6789 y 3210 … 9876). Cada letra se escribe como una etiqueta que reconoce la mayúscula o la minúscula, es decir, cuenta como un solo símbolo.

La composición P2 a P5 también se puede escribir como una única ER, unión de las 4! = 24 permutaciones de las cuatro clases:

```
COMP = Σ*LΣ*lΣ*NΣ*SΣ* | Σ*LΣ*lΣ*SΣ*NΣ* | ... | Σ*SΣ*NΣ*lΣ*LΣ*
```

Reconoce el mismo lenguaje que P2 ∩ P3 ∩ P4 ∩ P5 (comprobado con el programa). Su AFD mínimo tiene 16 estados, uno por cada subconjunto de clases ya vistas, pero el método del árbol genera 12 034 estados antes de minimizar; por eso el diseño usa P2 a P5 por separado.

Correos (at = A | N | H | E ; X = at | P)

```
C1  X+ @ X+
C2  at+ ( P at+ )*
C3  X{1,64}
C4  et ( P et )+          con et = (A|N) ( (A|N|H){0,61} (A|N) )?
C5  (A|N|H|P)* P A A+
C6  X{1,255}
```

C2 y C3 se aplican a la parte local; C4, C5 y C6, al dominio (ver 4.c). Sin las cotas de longitud, todo el correo se resume en una sola ER, que es la que se usa para el diagrama final:

```
CORREO = at+ ( P at+ )* @ ( et P )+ A A+
         con et = (A|N) ( (A|N|H)* (A|N) )?
```

4.d (complemento) Gramática regular y derivación sintáctica

Este complemento atiende el objetivo de aplicar gramáticas regulares y derivación sintáctica. Se ubica aquí porque la gramática es la tercera forma de describir el mismo lenguaje regular (ER, gramática, autómata).

Fundamento

Una gramática regular lineal por la derecha es G = (N, Σ, P, S) donde toda producción tiene la forma A → aB, A → a o A → ε. Equivale a un autómata: cada estado es un no terminal, cada transición δ(A, a) = B produce A → aB, y cada estado de aceptación A produce A → ε. Una cadena pertenece al lenguaje si existe una derivación S ⇒ … ⇒ w, donde cada paso aplica una producción. En las producciones, una

etiqueta representa una producción por cada símbolo del grupo; por ejemplo, `S → at L` significa S → xL para cada x de los 81 símbolos de `at`.

G_P1: longitud (P1)

No terminales A0, A1, …, A64; símbolo inicial S = A0.

```
Ai → x A(i+1)    para todo x ∈ Σ, 0 ≤ i ≤ 63
Ai → ε           para 15 ≤ i ≤ 64
```

G_COMP: composición (P2 a P5)

Un no terminal `Q_X` por cada subconjunto X de {L, l, N, S} (16 en total). `Q_X` significa «ya se vieron las clases de X». El símbolo inicial es `Q_∅`.

```
Q_X → c Q_(X ∪ clase(c))    para todo c ∈ Σ
Q_{L,l,N,S} → ε
```

Derivación de `Aa1!` (cumple las cuatro clases):

```
Q_∅ ⇒ A Q_{L} ⇒ A a Q_{L,l} ⇒ A a 1 Q_{L,l,N} ⇒ A a 1 ! Q_{L,l,N,S} ⇒ Aa1!
```

Si la cadena es `Aa1`, la derivación llega a `Q_{L,l,N}`, que no tiene producción hacia ε: no es derivable, falta el especial (P5). Como `Aa1!` tiene solo 4 símbolos, G_P1 no la deriva (P1 falla); una credencial se acepta solo si todas las gramáticas de aceptación la derivan.

G_P6: tres símbolos idénticos consecutivos (regla de rechazo)

```
S → c S          para todo c ∈ Σ
S → x X1x        para todo x ∈ Σ
X1x → x X2x
X2x → x T
T → c T | ε
```

Derivación de `Aaaa1` (la cadena se reconoce, es decir, se marca como insegura):

```
S ⇒ A S ⇒ A a X1a ⇒ A a a X2a ⇒ A a a a T ⇒ A a a a 1 T ⇒ Aaaa1
```

G_CORREO: correo completo (ER CORREO sin cotas de longitud)

No terminales: S, L, M, D0, E, G, D, T1, T2. Terminales agrupados por etiquetas: `at` (A, N, H, E), `P` (punto), `@`, `A` (letras), `N` (números), `H` (guion).

```
S  → at L
L  → at L | P M | @ D0          (L: parte local, último símbolo es atext)
M  → at L                       (M: se leyó un punto, exige atext)
D0 → (A|N) E                    (primera etiqueta tras la arroba)
E  → (A|N) E | H G | P D        (E: etiqueta cuyo último símbolo es letra o número)
G  → H G | (A|N) E              (G: se leyó un guion; no puede cerrar etiqueta)
D  → (A|N) E | A T1             (etiqueta nueva tras un punto, o inicio del TLD)
T1 → A T2
T2 → A T2 | ε                   (TLD: solo letras, mínimo 2)
```

D0 y D son distintos porque tras la arroba no puede comenzar directamente el TLD: el dominio exige al menos un punto.

Derivación de `a.b@c-d.gt`:

```
S ⇒ a L                     (S → at L)
  ⇒ a . M                   (L → P M)
  ⇒ a . b L                 (M → at L)
  ⇒ a . b @ D0              (L → @ D0)
  ⇒ a . b @ c E             (D0 → (A|N) E)
  ⇒ a . b @ c - G           (E → H G)
  ⇒ a . b @ c - d E         (G → (A|N) E)
  ⇒ a . b @ c - d . D       (E → P D)
  ⇒ a . b @ c - d . g T1    (D → A T1)
  ⇒ a . b @ c - d . g t T2  (T1 → A T2)
  ⇒ a.b@c-d.gt              (T2 → ε)
```

Cadenas rechazadas, vistas como derivaciones que se bloquean:

- `a..b@c.gt`: S ⇒ a L ⇒ a . M, y M solo tiene la producción M → at L; el segundo punto no es atext, así que no hay producción aplicable. Falla C2 (punto doble).
- `usuario@ejemplo`: la derivación llega a E después de `ejemplo` y E no tiene producción hacia ε ni se puede derivar el TLD. Falla C4 y C5 (sin punto ni TLD).
- `x@c-.gt`: tras el guion se está en G, que no tiene producción con punto. Falla C4 (guion al final de etiqueta).

4.d.i Convertir a AFND con transiciones vacías (construcción de Thompson)

Fundamento

El algoritmo de Thompson construye el AFND-ε N(r) de una expresión regular r por inducción sobre su estructura: primero los fragmentos de las hojas, luego se combinan con las operaciones de unión, concatenación y cierre de Kleene. Cada fragmento tiene un único estado inicial, sin transiciones entrantes, y un único estado final, sin transiciones salientes. Cada estado tiene a lo sumo una transición con símbolo o dos transiciones ε. Las abreviaturas se expanden a las tres operaciones básicas antes de construir.

Reglas de construcción

- **ε:** estado inicial i y final f con i –ε→ f. Usa 2 estados.
- **Símbolo o etiqueta a:** i –a→ f. Usa 2 estados. Una etiqueta es una sola arista que representa todos los símbolos del grupo.
- **Unión `s|t`:** se crean un inicial i y un final f nuevos. Se agregan i –ε→ inicial de s, i –ε→ inicial de t, final de s –ε→ f y final de t –ε→ f. Usa |s| + |t| + 2 estados.
- **Concatenación `st`:** se agrega final de s –ε→ inicial de t. El inicial es el de s y el final es el de t. Usa |s| + |t| estados.
- **Cierre `s*`:** se crean i y f nuevos. Se agregan i –ε→ inicial de s, i –ε→ f, final de s –ε→ inicial de s y final de s –ε→ f. Usa |s| + 2 estados.
- **Abreviaturas:** `s+` = `s s*` usa 2|s| + 2 estados; `s?` = `s|ε` usa |s| + 4 estados; `s{m,n}` = `s^m (s?)^(n−m)`; la unión de k etiquetas usa 4k − 2 estados (k = 4 da 14).

Las Figuras 1a a 1d ilustran estas reglas (símbolo, unión, concatenación y cierre).

Correspondencia con las reglas vistas en clase. El material de clase (Reglas de nomenclatura de Thompson) construye la concatenación `rs` compartiendo el estado intermedio (3 estados en total: el final

de `r` es el inicial de `s`) y construye `r+` directamente, con un lazo ε de regreso al inicio de `r` (|r| + 2 estados), como muestra la Figura 1e. Este documento usa una variante equivalente: une los fragmentos con una arista ε (|s| + |t| estados) y expande `s+ = s s*` (2|s| + 2 estados). Se eligió porque una sola fórmula por operador permite calcular todos los tamaños de las tablas de 4.d.ii. Ambas variantes reconocen el mismo lenguaje; con las reglas de clase el AFND-ε tiene menos estados (por ejemplo, `a+` usa 4 en lugar de 6), y el algoritmo de subconjuntos produce el mismo AFD mínimo.

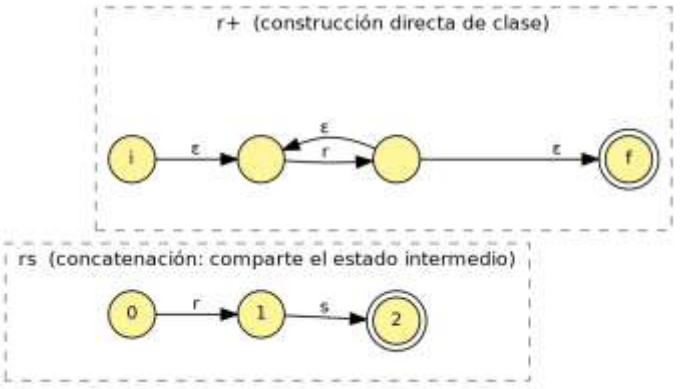


Figura 1e. Reglas de clase: concatenación rs (estado compartido) y r+ directo.

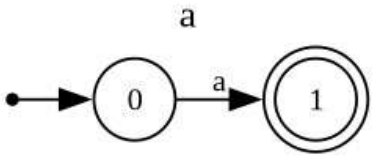


Figura 1a. Fragmento de Thompson: símbolo.

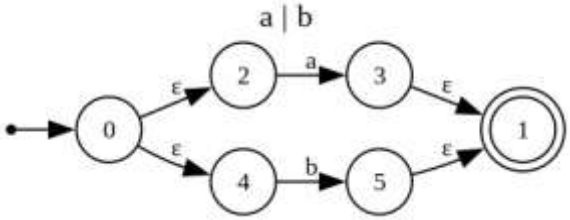


Figura 1b. Fragmento de Thompson: unión.

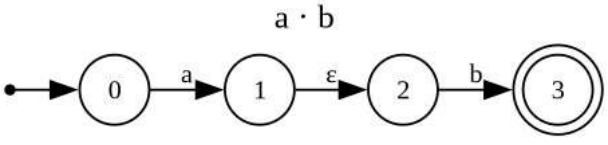


Figura 1c. Fragmento de Thompson: concatenación.

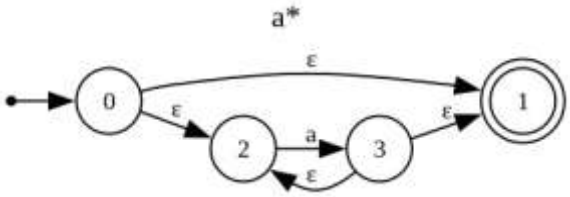


Figura 1d. Fragmento de Thompson: cierre de Kleene.

Ejemplo 1: parte local `a+ (. a+)*` (etiqueta a = atext)

Expandida: `a a* ( . a a* )*`. Se construye de adentro hacia afuera:

Paso	Fragmento	Estados nuevos	Transiciones agregadas
1	`a` (primer átomo)	0, 1	0 –a→ 1
2	`a` para el cierre	2, 3	2 –a→ 3
3	`a*` del paso 2	4, 5	4 –ε→ 2, 4 –ε→ 5, 3 –ε→ 2, 3 –ε→ 5
4	`a+` = `a · a*`	ninguno	1 –ε→ 4
5	`.`	6, 7	6 –.→ 7
6	`a`	8, 9	8 –a→ 9
7	`a` para el cierre	10, 11	10 –a→ 11
8	`a*` del paso 7	12, 13	12 –ε→ 10, 12 –ε→ 13, 11 –ε→ 10, 11 –ε→ 13
9	`. a a*`	ninguno	7 –ε→ 8, 9 –ε→ 12
10	`(. a a*)*`	14, 15	14 –ε→ 6, 14 –ε→ 15, 13 –ε→ 6, 13 –ε→ 15
11	`a+ (. a+)*`	ninguno	5 –ε→ 14

Resultado: 16 estados, inicial 0, final 15 (6 estados de `a+` más 10 del cierre). Tabla de transiciones:

Estado	a	.	ε
0	1	—	—
1	—	—	4
2	3	—	—
3	—	—	2, 5
4	—	—	2, 5
5	—	—	14
6	—	7	—
7	—	—	8
8	9	—	—
9	—	—	12
10	11	—	—
11	—	—	10, 13
12	—	—	10, 13
13	—	—	6, 15
14	—	—	6, 15
15 (final)	—	—	—

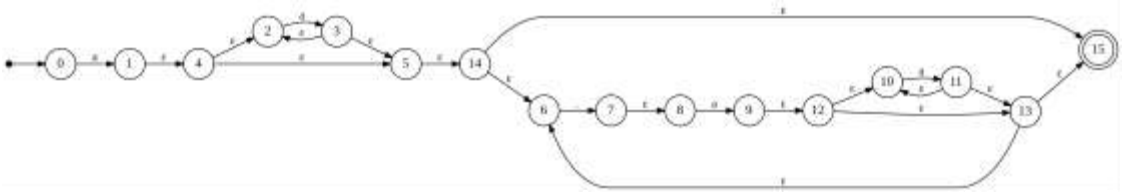


Figura 2. AFND-ε de la parte local a+(.a+), con la numeración de la tabla anterior.*

Simulación con ε-clausuras:

- Inicio: ε-clausura({0}) = {0}.
- Leer `a`: mover = {1}; ε-clausura = {1, 2, 4, 5, 6, 14, 15}. Contiene 15, así que `a` se acepta.
- Leer `.`: mover = {7}; ε-clausura = {7, 8}. No contiene 15, así que `a.` se rechaza (punto final).
- Leer `a`: mover = {9}; ε-clausura = {6, 9, 10, 12, 13, 15}. Contiene 15, así que `a.a` se acepta.
- Para `a..a`: después de `a.` el conjunto es {7, 8}; con un segundo `.` el mover es vacío y la cadena se rechaza (punto doble).

Ejemplo 2: composición `Σ* L Σ*` (regla P2), con Σ = L|l|N|S

- Σ es la unión de 4 etiquetas: 4 hojas de 2 estados más 3 uniones de 2 estados da 14 estados.
- `Σ*` agrega 2 estados: 16.
- `Σ* L Σ*` es la concatenación de 16 + 2 + 16 = 34 estados (inicio 0, final 19 en la numeración de la Figura 3).

P3, P4 y P5 tienen la misma estructura y también 34 estados; solo cambia la etiqueta central.

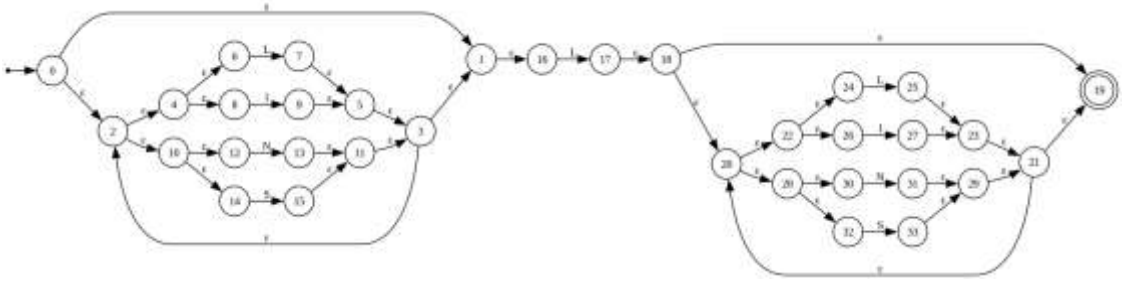


Figura 3. AFND-ε de P2 (inicio: 0, final: 19).

Ejemplo 3: correo completo

ER: `at+ ( P at+ )* @ ( et P )+ A A+`, con `et = (A|N) ( (A|N|H)* (A|N) )?` y `at = A|N|H|E`.
Tamaño de cada subexpresión, de adentro hacia afuera:

- `at` (4 etiquetas): 14 estados
- `at+`: 30
- `P at+`: 32
- `(P at+)*`: 34
- Parte local `at+ (P at+)*`: 30 + 34 = 64
- `@`: 2
- `(A|N)`: 6

- `(A|N|H)`: 10; `(A|N|H)*`: 12
- `(A|N|H)* (A|N)`: 12 + 6 = 18
- `((A|N|H)* (A|N))?`: 22
- `et`: 6 + 22 = 28
- `et P`: 30; `(et P)*`: 32; `(et P)+`: 30 + 32 = 62
- `A`: 2; `A+`: 6
- **CORREO completo:** 64 + 2 + 62 + 2 + 6 = **136 estados**

Verificación de los tamaños de la tabla

Cada tamaño de la columna «AFND-ε (Thompson)» de la tabla de 4.d.ii se reproduce con las fórmulas de arriba (Σ de contraseñas = 4 etiquetas = 14 estados; X = at|P = 5 etiquetas = 18 estados):

ER	Cálculo	Estados
P1 `Σ{15,64}`	15 · 14 + 49 · (14 + 4) = 210 + 882	1092
P2 `Σ* L Σ*`	16 + 2 + 16	34
P6 repetición	94 alternativas de 3 símbolos (94 · 6) + 93 uniones (93 · 2) + 2 · 16 = 564 + 186 + 32	782
P7 secuencias	60 palabras de 4 símbolos (60 · 8) + 59 uniones (59 · 2) + 32 = 480 + 118 + 32	630
P8 palabras	51 letras (51 · 2) + 6 uniones (6 · 2) + 32 = 102 + 12 + 32	146
C1 `X+ @ X+`	38 + 2 + 38	78
C2 parte local	30 + 34	64
C3 `X{1,64}`	18 + 63 · (18 + 4) = 18 + 1386	1404
C4 etiquetas	et con cotas = 6 + (61 · 14 + 6 + 4) = 870; `(P et)+` = 2 · 872 + 2 = 1746; total 870 + 1746	2616
C5 TLD	16 + 2 + 2 + 6	26
C6 `X{1,255}`	18 + 254 · 22	5606
CORREO	ver Ejemplo 3	136

Notas sobre la tabla:

- En P8 las 7 palabras suman 51 letras: password (8), contrasena (10), admin (5), qwerty (6), iloveyou (8), welcome (7), letmein (7).
- Las ER de contraseñas operan sobre las etiquetas L, l, N, S, que es lo que da los tamaños de arriba. P6 es la excepción, porque enumera una alternativa `xxx` por cada símbolo x de Σ.

4.d.ii Aplicar el algoritmo de Thompson y del árbol para convertirlo a AFD

Este inciso convierte a AFD por las dos rutas que pide la guía. La ruta de Thompson parte del AFND-ε construido en 4.d.i y lo determiniza con el algoritmo de subconjuntos. La ruta del árbol parte directamente de la ER, sin pasar por el AFND-ε. Ambas producen AFD equivalentes; aquí se presenta primero el método del árbol y después la ruta de Thompson.

Ruta del árbol: método del árbol (SiguientePos)

Se aumenta la ER con un marcador final `#`, se numera cada hoja (posición) y se calculan anulable, PrimeraPos, ÚltimaPos y SiguientePos con estas reglas:

- anulable(ε) = verdadero; anulable(hoja) = falso.
- Unión: PrimeraPos y ÚltimaPos son la unión de los de los hijos.
- Concatenación c1·c2: PrimeraPos(c1) se amplía con PrimeraPos(c2) solo si c1 es anulable; ÚltimaPos(c2) se amplía con ÚltimaPos(c1) solo si c2 es anulable.
- Cierre x*: PrimeraPos y ÚltimaPos son los de x.
- SiguientePos: en c1·c2, cada posición de ÚltimaPos(c1) recibe PrimeraPos(c2); en x*, cada posición de ÚltimaPos(x) recibe PrimeraPos(x).

Los estados del AFD son conjuntos de posiciones. El inicial es PrimeraPos de la raíz; desde un estado S con el símbolo a se llega a la unión de SiguientePos(p) para las posiciones $p \in S$ etiquetadas con a. Es de aceptación todo estado que contiene la posición de `#`.

Correspondencia con la guía de clase (ER a AFD). Los nombres en inglés se escriben aquí como en clase: nullable = Anulable, firstpos = PrimeraPos, lastpos = ÚltimaPos y followpos = SiguientePos. El procedimiento por ejemplo es: (1) dibujar el árbol de la ER con el símbolo terminal `#`; (2) numerar las hojas y marcar si cada nodo es anulable; (3) calcular PrimeraPos y ÚltimaPos de cada nodo; (4) calcular SiguientePos de cada hoja; (5) armar la tabla de estados D, empezando por la PrimeraPos de la raíz y uniendo la SiguientePos de las posiciones etiquetadas con cada símbolo; (6) renombrar los conjuntos (A, B, C…). Si la ER usa los operadores `+` y `?`, la guía indica: Anulable(s+) = Anulable(s) y Anulable(s?) = verdadero, con PrimeraPos y ÚltimaPos iguales a las de s; en este documento ambos se expanden antes (s+ = s s*, s? = s|ε), por lo que el árbol solo contiene unión, concatenación y cierre. En los árboles de las figuras, cada nodo muestra A (anulable: V o F), P (PrimeraPos) y U (ÚltimaPos).

Ejemplo 1: P2 = Σ* L Σ*

Posiciones: 1 a 4 son las etiquetas L, l, N, S del primer Σ*; 5 es la etiqueta L central; 6 a 9 son las etiquetas del segundo Σ*; 10 es #.

Árbol de la ER aumentada Σ* L Σ* # con hojas numeradas y los valores Anulable, PrimeraPos y ÚltimaPos de cada nodo (Figura 4a). La raíz tiene PrimeraPos = {1,2,3,4,5}, que será el estado inicial.

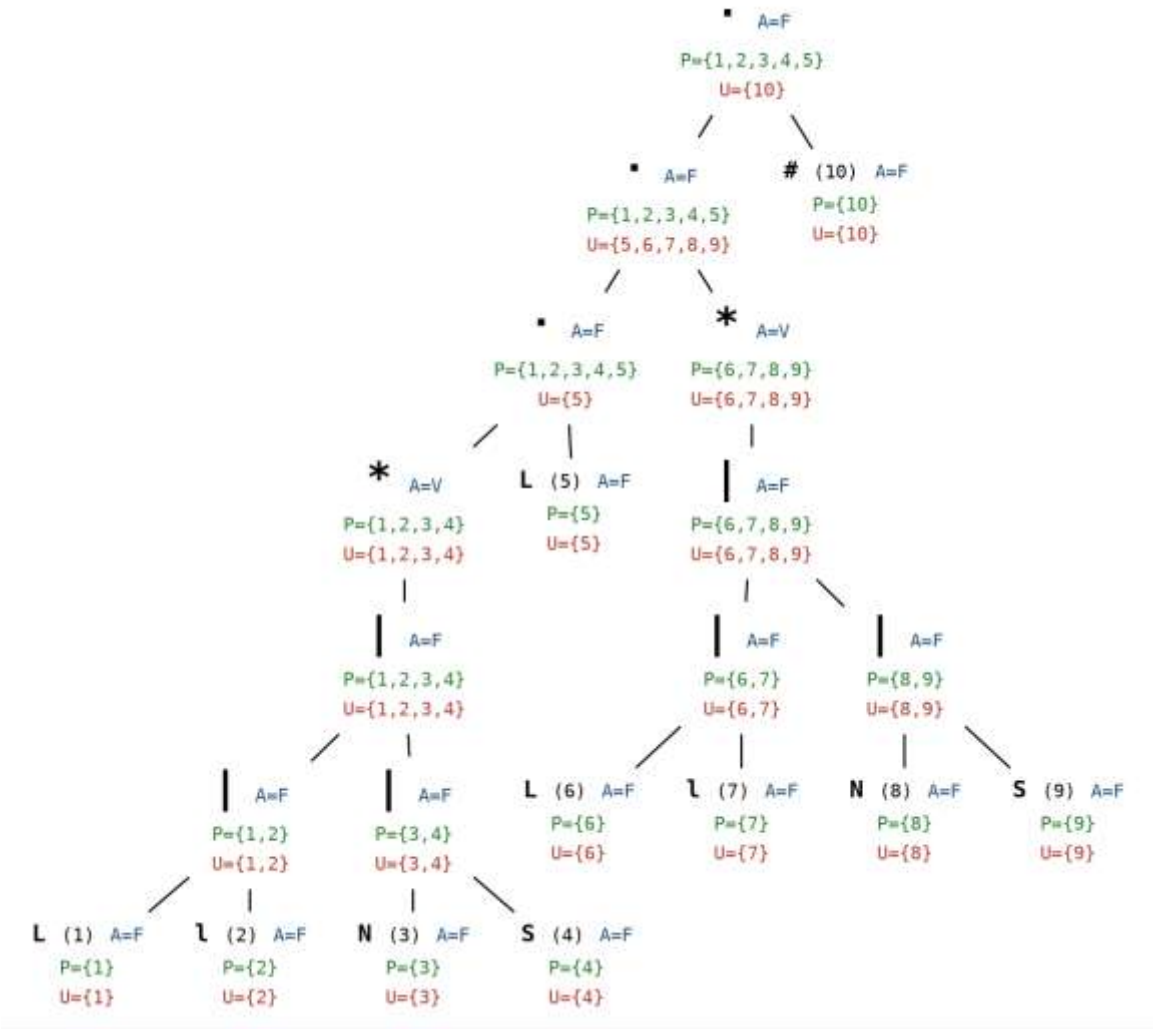


Figura 4a. Árbol de P2 = Σ L Σ* # con Anulable (A), PrimeraPos (P) y ÚltimaPos (U).*

Tabla SiguientePos (calculada con las dos reglas de la guía):

Posición	Etiqueta	SiguientePos
1	L	{1,2,3,4,5}
2	l	{1,2,3,4,5}
3	N	{1,2,3,4,5}
4	S	{1,2,3,4,5}
5	L	{6,7,8,9,10}
6	L	{6,7,8,9,10}
7	l	{6,7,8,9,10}
8	N	{6,7,8,9,10}
9	S	{6,7,8,9,10}
10	#	∅

PrimeraPos(raíz) = {1,2,3,4,5} = A. Desde A con L se alcanza SiguientePos(1) ∪ SiguientePos(5) = {1,…,10} = B; con l, N o S se queda en A. Desde B, cualquier símbolo regresa a B. B contiene la posición 10 (#), por lo tanto es de aceptación (Figura 4: 2 estados).

Tabla de estados D de P2 (Estado inicial: A; aceptación: los que contienen la posición 10, es decir #):

Conjunto	Estado	L	l	N	S	Aceptación
{1,2,3,4,5}	A	B	A	A	A	No
{1,…,10}	B	B	B	B	B	Sí

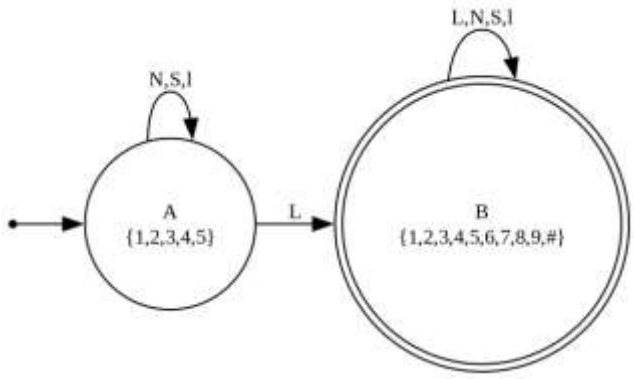


Figura 4. AFD de P2 obtenido con el método del árbol (2 estados; B es de aceptación).

Ejemplo 2: parte local `a+ ( . a+ )*`

Expandida como `a a* ( . a a* )* #`: posición 1 es la `a` inicial, 2 la `a` del cierre, 3 el punto, 4 la `a` tras el punto, 5 la `a` del cierre interno y 6 es `#`.

Árbol de `a a* ( . a a* )* #` con los valores de cada nodo (Figura 5a). Las posiciones 1 a 5 son las hojas `a`, `.`, `a`, `a` y la 6 es `#`.

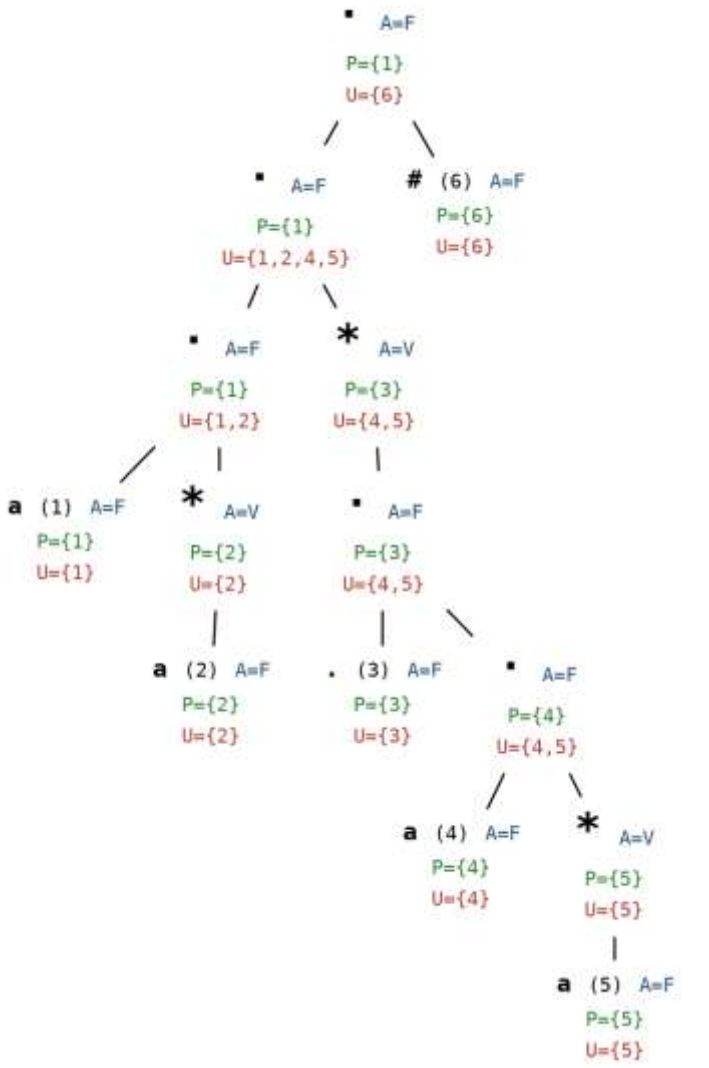


Figura 5a. Árbol de la parte local con Anulable (A), PrimeraPos (P) y ÚltimaPos (U).

Posición	Etiqueta	SiguientePos
1	a	{2,3,6}
2	a	{2,3,6}
3	.	{4}
4	a	{3,5,6}
5	a	{3,5,6}
6	#	∅

A = PrimeraPos = {1}. A con a: {2,3,6} = B (aceptación, contiene 6). B con a: B; B con «.»: {4} = C. C con a: {3,5,6} = D (aceptación). D con a: D; D con «.»: C. Al minimizar, A ≡ C y B ≡ D, quedando dos estados: «esperar átomo» y «átomo leído» (Figura 5).

Tabla de estados D de la parte local (inicial: A; aceptación: los que contienen la posición 6):

Conjunto	Estado	a	.	Aceptación
{1}	A	B	—	No
{2,3,6}	B	B	C	Sí
{4}	C	D	—	No
{3,5,6}	D	D	C	Sí

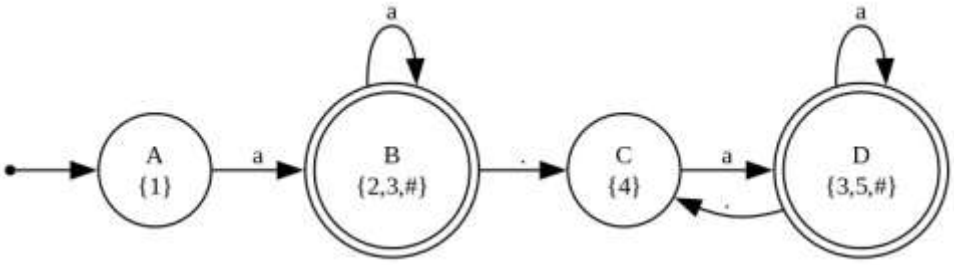


Figura 5. AFD de la parte local por el método del árbol.

Ejemplo 3: correo completo

Con la ER CORREO se obtiene un AFD de 12 estados (8 al minimizar), mostrado en la Figura 6. Las aristas están rotuladas con las etiquetas; A es el estado inicial y K el único de aceptación. Los estados A, B, C, L y D reconocen la parte local y la arroba (L es el estado al que se llega tras un punto y un átomo); los estados E a K reconocen el dominio. E acepta letras y números, G es «acaba de leer un guion» (no puede terminar una etiqueta ni seguir de punto), F es «acaba de leer un punto», y K solo se alcanza cuando la última etiqueta contiene únicamente letras.

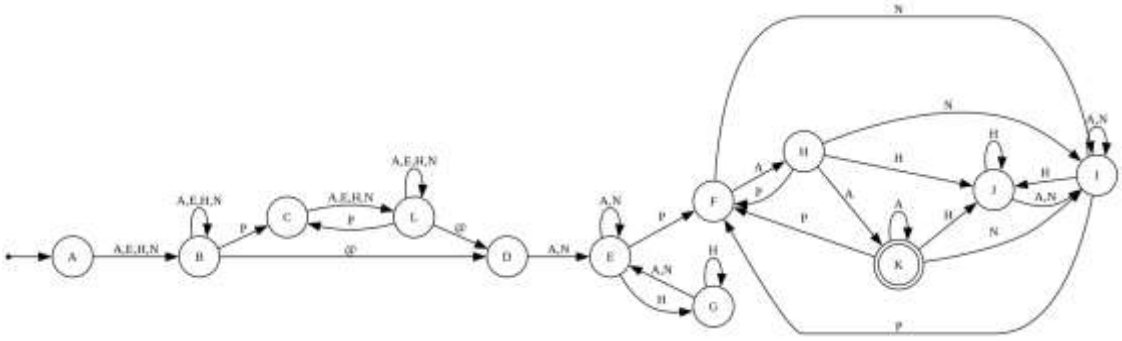


Figura 6. AFD del correo completo (método del árbol, 12 estados).

AFD mínimo (8 estados), con las clases de símbolos A (letras), N (números), H (guion), E (otros atext), P (punto) y @:

Estado	Significado	A	N	H	E	P	@
q0 (inicial)	inicio de la parte local o tras un punto: espera atext	q1	q1	q1	q1	—	—
q1	parte local, último símbolo atext	q1	q1	q1	q1	q0	q2
q2	se leyó la arroba: espera letra o número	q3	q3	—	—	—	—
q3	etiqueta que acaba en letra o número	q3	q3	q4	—	q5	—
q4	se leyó un guion	q3	q3	q4	—	—	—
q5	se leyó un punto del dominio	q6	q3	—	—	—	—
q6	tras el punto, una sola letra	q7	q3	q4	—	q5	—
q7 (aceptación)	tras el punto, solo letras (mínimo 2)	q7	q3	q4	—	q5	—

Este autómata coincide con el de la gramática G_CORREO del complemento de 4.d (q0 reúne a S y M; q1 es L; q2 es D0; q3 es E; q4 es G; q5 es D; q6 y q7 son los conjuntos {E, T1} y {E, T2}). Ejemplos: `usuario@ejemplo.com` termina en q7 y se acepta; `usuario@ejemplo` termina en q3 y se rechaza; `ana@ejemplo.c0m` termina en q3 y se rechaza (el TLD no es solo letras).

Ruta de Thompson: algoritmo de subconjuntos sobre el AFND de 4.d.i

El algoritmo parte del AFND-ε de Thompson: el estado inicial es la ε-clausura del inicial y cada transición es la ε-clausura de mover(S, a). Aplicado al AFND de la parte local (Ejemplo 1 de 4.d.i, etiqueta a):

Estado AFD	Conjunto de estados del AFND	a	.	Aceptación
S0	{0}	S1	—	no
S1	{1, 2, 4, 5, 6, 14, 15}	S2	S3	sí
S2	{2, 3, 5, 6, 14, 15}	S2	S3	sí
S3	{7, 8}	S4	—	no
S4	{6, 9, 10, 12, 13, 15}	S5	S3	sí
S5	{6, 10, 11, 13, 15}	S5	S3	sí

Un estado es de aceptación si contiene el final del AFND (15). Al minimizar, S0 ≡ S3 y S1 ≡ S2 ≡ S4 ≡ S5, y se obtienen los mismos dos estados del método del árbol. Con las cuatro etiquetas de `at` (A, N, H, E) cada uno de los cuatro estados S1, S2, S4 y S5 se repite por etiqueta, lo que da $2 + 4 \cdot 4 = 18$ estados, el valor de la columna «AFD subconjuntos» de C2.

Tamaños obtenidos para todas las reglas (los tres algoritmos coinciden en el lenguaje reconocido):

ER	Hojas	AFND-ε (Thompson)	AFD subconjuntos	AFD árbol	AFD mínimo
P1 `Σ{15,64}`	256	1092	257	65	65
P2 `Σ*LΣ*`	9	34	9	2	2
P6 repetición	290	782	565	377	190
P7 secuencias	248	630	763	314	158
P8 palabras	59	146	213	90	44
COMP (24 perm.)	576	2158	48137	12034	16
C1 estructura	21	78	22	4	4
C2 parte local	17	64	18	4	2
C3 local ≤ 64	320	1404	321	65	65
C4 etiquetas	563	2616	564	374	250
C5 TLD	8	26	8	4	4
C6 dominio ≤ 255	1275	5606	1276	256	256
CORREO completo	37	136	33	12	8

P3, P4 y P5 tienen exactamente el mismo tamaño que P2 (2 estados en el AFD), porque solo cambia la etiqueta central. La ER de P1 produce un contador lineal de 65 estados (0 a 64 símbolos leídos). Ambos métodos se implementaron y producen AFD equivalentes en las 16 ER (verificado por búsqueda en pares de estados); en estas ER el método del árbol da un AFD más compacto porque no arrastra los estados intermedios de Thompson.

Anexos del paso 4

Anexo A. Verificación con casos de prueba

Los AFD construidos se contrastaron con implementaciones independientes de cada regla (expresiones de Python y comprobaciones directas) sobre 1 500 cadenas aleatorias por regla, sin discrepancias, y la ER de composición de 24 permutaciones se contrastó con P2 ∩ P3 ∩ P4 ∩ P5. A continuación, el reporte por regla que produce el validador sobre credenciales de ejemplo.

Contraseñas

Entrada	Resultado	Reglas incumplidas
`Pl4t0n0#Guatemala2026`	Válida	—
`Mi␣perro␣come␣7␣Huesos!`	Válida	—
`Xk9$mTq2#vLw8@Rz`	Válida	—
`Vj7!nQe4%Ha2&Ub6(Kd`	Válida	—
`H7$kP2#mQ9@xL4!zR8`	Válida	—
`␣␣␣␣Segura2026#xyz`	Inválida	P1: longitud = 14
`Abc123!`	Inválida	P1: longitud = 7
`contrasenasegura`	Inválida	P2: falta mayuscula; P4: falta digito; P5: falta caracter especial; P8: contiene palabra prohibida
`CONTRASENA2026!`	Inválida	P3: falta minuscula; P8: contiene palabra prohibida
`Password12345!!`	Inválida	P7: secuencia ascendente/descendente de 4; P8: contiene palabra prohibida
`Aaaa1111!!!!bbbb`	Inválida	P6: 3+ caracteres identicos consecutivos
`Tr0ub4dor&3xYz!qW9`	Válida	—
`abcd1234EFGH!@#$`	Inválida	P7: secuencia ascendente/descendente de 4
`Zq8#Lp3$Mn5@Wx1^Yt`	Válida	—
`Clave#Segura7Ñandú9`	Inválida	P0: simbolo no permitido en pos. 14

Correos

Entrada	Resultado	Reglas incumplidas
`cristian.lopez@ejemplo.com`	Válido	—
`usuario+etiqueta@correo.edu.gt`	Válido	—
`a_b-c'd!e@sub.dominio.org`	Válido	—
`.usuario@ejemplo.com`	Inválido	C2: dot-atom invalido (punto inicial/final/doble)
`usuario.@ejemplo.com`	Inválido	C2: dot-atom invalido (punto inicial/final/doble)
`usuario..nombre@ejemplo.com`	Inválido	C2: dot-atom invalido (punto inicial/final/doble)
`usuarioejemplo.com`	Inválido	C1: 2 arrobas
`usuario@ejemplo`	Inválido	C4: etiqueta invalida (guion en borde, vacia o >63) o sin punto; C5: TLD debe ser solo letras y >= 2
`usuario@-ejemplo.com`	Inválido	C4: etiqueta invalida (guion en borde, vacia o >63) o sin punto
`usuario@ejemplo..com`	Inválido	C4: etiqueta invalida (guion en borde, vacia o >63) o sin punto
`usuario␣ejemplo@correo.com`	Válido	—
`usuario@ejemplo.c`	Inválido	C5: TLD debe ser solo letras y >= 2
`ana@ejemplo.c0m`	Inválido	C5: TLD debe ser solo letras y >= 2
`ana@ejem_plo.com`	Inválido	C4: etiqueta invalida (guion en borde, vacia o >63) o sin punto; C5: TLD debe ser solo letras y >= 2
`aaaa…aaaa@ejemplo.com` (parte local de más de 64 `a`)	Inválido	C3: local > 64
`ana@bbbb…bbbb.com` (etiqueta de más de 63 `b`)	Inválido	C4: etiqueta invalida (guion en borde, vacia o >63) o sin punto

Nota: «␣» marca un espacio en la entrada. La frase «Mi perro come 7 Huesos!» se evalúa como «Miperrocome7Huesos!» al descartar los espacios. «Password12345!!» se rechaza por P8 (palabra prohibida) y por P7 (secuencia 2345). Del mismo modo, «usuario ejemplo@correo.com» se evalúa como «usuarioejemplo@correo.com» y resulta válido, porque el enunciado indica que los espacios no se toman en cuenta; con las reglas estrictas de RFC 5322 sería inválido.

Aclaración: en `ana@ejem_plo.com` el TLD `com` es correcto; C5 se reporta porque su ER se evalúa sobre todo el dominio y el símbolo _ no pertenece a (A|N|H|P). La causa real del rechazo es C4.

Anexo B. Alcance y limitaciones

- **Correos.** No se validan direcciones con comillas, IP literal, comentarios ni Unicode; son formas válidas en RFC 5322 pero no las acepta ningún servicio público (paso 3). Tampoco se aceptan dominios de nivel superior internacionalizados codificados en Punycode (por ejemplo, `xn--p1ai`), porque la regla C5 exige un TLD formado solo por letras.
- **Contraseñas.** P8 usa una lista finita de palabras. Las variantes con sustitución de caracteres (`p4ssw0rd`) y la comparación con contraseñas filtradas no son expresables con un autómata finito.
- **Diseño.** La ER general de la política requiere intersección de lenguajes; se resuelve con un AFD por regla y un autómata producto, lo que además da el reporte de la regla que falla.

Anexo C. Criterio propuesto de nivel de seguridad (punto 7.a.iv)

Este criterio es una propuesta de diseño propia para la etapa de la aplicación; no proviene de NIST ni de CIS. Se apoya en la idea de ambos estándares de que la longitud es el factor que más pesa, y se calcula sobre lo que ya reporta el validador.

Nivel	Condición	Entropía bruta estimada
Baja	Incumple alguna regla de aceptación (P0 a P5) o coincide con algún filtro de rechazo (P6 a P8)	No es una contraseña aceptable
Media	Cumple todas las reglas y tiene de 15 a 19 caracteres	Entre 98 y 124 bits (estimación bruta)
Fuerte	Cumple todas las reglas y tiene 20 o más caracteres	131 bits o más (estimación bruta)

La entropía bruta se estima como $n \cdot \log_2(94) \approx 6.55$ bits por símbolo, porque una contraseña que pasa todas las reglas contiene las cuatro clases y su alfabeto efectivo es de 94 símbolos. Es un límite superior, válido solo si los símbolos se eligieron al azar: una contraseña ideada por una persona tiene menos entropía real. Por eso el filtro de rechazo (P6 a P8) tiene prioridad y el nivel Baja se asigna antes de calcular nada.

Ejemplos con el validador: `Pl4t0n0#Guatemala2026` (21 caracteres) es Fuerte; `Miperrocome7Huesos!` (19) es Media; `Xk9$mTq2#vLw8@Rz` (16) es Media; `Abc123!` es Baja (incumple P1). Los umbrales de 15 y 20 caracteres son ajustables; 15 es el mínimo de NIST Rev. 4 para contraseñas de un solo factor, y 20 es una marca práctica para frases largas. El generador del punto 7.a.iii podrá apuntar a longitudes de 20 o más para producir contraseñas Fuertes.

Referencias

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.ª ed.). Addison-Wesley.

Center for Internet Security. (2021). *CIS Password Policy Guide*, versión 21.12. https://www.cisecurity.org

Crocker, D. y Overell, P. (2008). *RFC 5234: Augmented BNF for Syntax Specifications: ABNF*. Internet Engineering Task Force.

Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3.ª ed.). Pearson/Addison-Wesley.

Klensin, J. (2004). *RFC 3696: Application Techniques for Checking and Transformation of Names*. Internet Engineering Task Force.

Klensin, J. (2008). *RFC 5321: Simple Mail Transfer Protocol*. Internet Engineering Task Force.

Mockapetris, P. (1987). *RFC 1035: Domain Names — Implementation and Specification*. Internet Engineering Task Force.

NIST. (2025). *NIST Special Publication 800-63B Revision 4: Digital Identity Guidelines — Authentication and Lifecycle Management*. National Institute of Standards and Technology. https://pages.nist.gov/800-63-4/sp800-63b.html

PCI Security Standards Council. (2024). *Payment Card Industry Data Security Standard: Requirements and Testing Procedures, v4.0.1*. https://www.pcisecuritystandards.org

Resnick, P. (Ed.). (2008). *RFC 5322: Internet Message Format*. Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc5322/

Yao, J. y Mao, W. (2012). *RFC 6531: SMTP Extension for Internationalized Email*. Internet Engineering Task Force.